

# **NEWSVAULT**

A Minor Project Report

Submitted in Partial Fulfilment of the Requirement for the Award of the  
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &  
ENGINEERING**

BY

**Laxman Singh Rajput**

**EN23CS301562**

Under the Guidance of

**Dr. Mahesh Patidar**



**Department of Computer Science & Engineering  
Faculty of Engineering**

**MEDICAPS UNIVERSITY, INDORE – 453331**

**APRIL – 2026**

## Report Approval

The project work "NEWSVAULT" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

### Internal Examiner

Name: \_\_\_\_\_

Designation: \_\_\_\_\_

Affiliation: \_\_\_\_\_

### External Examiner

Name: \_\_\_\_\_

Designation: \_\_\_\_\_

Affiliation: \_\_\_\_\_

## Declaration

I hereby declare that the project entitled "NEWSVAULT" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of Dr. Mahesh Patidar, Faculty of Engineering, Mediacaps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature: \_\_\_\_\_

Name: Laxman Singh Rajput

Roll No: EN23CS301562

Date: \_\_\_\_\_

## Certificate

I, Dr. Mahesh Patidar, certify that the project entitled "NEWSVAULT" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Laxman Singh Rajput (EN23CS301562) is the record carried out by him under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

---

Dr. Mahesh Patidar

Department of Computer Science & Engineering

Medicaps University, Indore

---

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

## Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale.

I also thank Prof. (Dr.) Ratnesh Litoriya, Associate Dean, Faculty of Engineering, Mediacaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department Prof. Dr. Kailash Chandra Bandhu for his continuous encouragement for the betterment of the project.

I am deeply grateful to my project guide Dr. Mahesh Patidar for his invaluable guidance, constant support, and constructive feedback throughout the development of this project. His expertise in cloud technologies and software engineering significantly shaped the direction of this work.

Without their collective help and support, this report would not have been possible.

**Laxman Singh Rajput**

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

## Abstract

NewsVault is a cloud-based digital newspaper subscription and distribution platform developed using the MERN stack (MongoDB, Express.js, React.js, Node.js) and deployed on Amazon Web Services (AWS) infrastructure. The platform addresses the key limitations of traditional newspaper subscription systems—namely manual record maintenance, cash-based payments, lack of digital accessibility, and poor scalability—by providing a centralized, automated, and highly available digital ecosystem.

The system enables subscribers to browse multiple newspaper publications, manage their subscriptions through an interactive dashboard, and access e-paper content via an integrated browser-based PDF reader. Content delivery is optimized using Amazon CloudFront CDN, while all newspaper PDFs are securely stored in Amazon S3 with time-limited Signed URL access control. Authentication and authorization are handled using JSON Web Tokens (JWT) with role-based access for Subscribers and Administrators.

The deployment pipeline follows modern DevOps practices: the application is containerized using Docker and deployed automatically to an AWS EC2 instance via a Jenkins CI/CD pipeline integrated with GitHub Webhooks. Nginx serves as the reverse proxy for SSL termination and load management.

The project demonstrates the integration of full-stack web development with cloud infrastructure and automated deployment to produce a production-ready, scalable media distribution platform.

**Keywords:** MERN Stack, Cloud Computing, AWS, Amazon S3, Amazon CloudFront, Jenkins, CI/CD, Docker, JWT Authentication, Digital Newspaper, Subscription Platform, Node.js, React.js, MongoDB, Nginx.

## Table of Contents

| Chapter / Section | Title                                 | Page No. |
|-------------------|---------------------------------------|----------|
|                   | Report Approval                       | ii       |
|                   | Declaration                           | iii      |
|                   | Certificate                           | iv       |
|                   | Acknowledgements                      | v        |
|                   | Abstract                              | vi       |
|                   | Table of Contents                     | vii      |
|                   | List of Figures                       | viii     |
|                   | List of Tables                        | ix       |
|                   | Abbreviations                         | x        |
| Chapter 1         | Introduction                          | 1        |
| 1.1               | Introduction                          | 1        |
| 1.2               | Literature Review                     | 1        |
| 1.3               | Problem Definition                    | 2        |
| 1.4               | Objectives                            | 2        |
| 1.5               | Methodology                           | 3        |
| 1.6               | Chapter Scheme                        | 3        |
| Chapter 2         | Requirements Specification            | 5        |
| 2.1               | Product Scope and Perspective         | 5        |
| 2.2               | User Classes and Characteristics      | 5        |
| 2.3               | Functional Requirements               | 5        |
| 2.4               | Non-Functional Requirements           | 6        |
| 2.5               | Operating Environment                 | 6        |
| 2.6               | Design and Implementation Constraints | 7        |
| 2.7               | Assumptions and Dependencies          | 7        |
| 2.8               | External Interface Requirements       | 7        |

|           |                                          |    |
|-----------|------------------------------------------|----|
| Chapter 3 | System Design                            | 9  |
| 3.1       | Methodology and Analysis Model           | 9  |
| 3.2       | Use Case Diagram and Specification       | 10 |
| 3.3       | Entity-Relationship (ER) Model           | 11 |
| 3.4       | Data Flow Diagram (DFD)                  | 12 |
| 3.5       | Control Flow Diagram (CFD)               | 14 |
| 3.6       | State Transition Diagram                 | 15 |
| 3.7       | Architectural Design                     | 16 |
| 3.8       | Database Design                          | 18 |
| 3.9       | Component Design and Flowchart           | 19 |
| 3.10      | Interface Design                         | 20 |
| Chapter 4 | Implementation, Testing, and Maintenance | 22 |
| 4.1       | Technologies, Languages, IDEs, and Tools | 22 |
| 4.2       | CI/CD Pipeline and Deployment            | 23 |
| 4.3       | Testing Techniques and Test Plans        | 23 |
| 4.4       | Installation Instructions                | 24 |
| 4.5       | End User Instructions                    | 25 |
| Chapter 5 | Results and Discussions                  | 26 |
| 5.1       | User Interface Representation            | 26 |
| 5.2       | Module Descriptions                      | 27 |
| 5.3       | Backend and Database Representation      | 27 |
| Chapter 6 | Summary and Conclusions                  | 27 |
| Chapter 7 | Future Scope                             | 30 |
|           | Appendix A: Glossary                     | 31 |
|           | Appendix B: To Be Determined List        | 33 |
|           | Bibliography / References                | 34 |

## List of Figures

| Figure No. | Figure Title                            | Page No. |
|------------|-----------------------------------------|----------|
| Figure 2.1 | Use Case Diagram – NewsVault            | 10       |
| Figure 2.2 | Entity-Relationship Diagram – NewsVault | 13       |
| Figure 2.3 | Data Flow Diagram – NewsVault           | 13       |
| Figure 2.4 | Control Flow Diagram – NewsVault        | 14       |
| Figure 2.5 | State Transition Diagram – NewsVault    | 16       |
| Figure 3.1 | System Architecture Diagram – NewsVault | 17       |
| Figure 3.2 | System Flowchart – NewsVault            | 20       |

## List of Tables

| Table No. | Table Title                                     | Page No. |
|-----------|-------------------------------------------------|----------|
| Table 2.1 | Functional Requirements Summary                 | 5        |
| Table 2.2 | Non-Functional Requirements                     | 6        |
| Table 3.1 | Use Case Specification – Access Digital E-Paper | 10       |
| Table 3.2 | Data Dictionary for MongoDB Collections         | 18       |
| Table 3.3 | Primary UI Screen Descriptions                  | 20       |
| Table 4.1 | Technology Stack Summary                        | 22       |
| Table 4.2 | API Endpoint Summary                            | 22       |
| Table 4.3 | Test Cases – Authentication Module              | 23       |
| Table 4.4 | Test Cases – Subscription & Content Delivery    | 24       |

## Abbreviations

| Abbreviation | Full Form                                      |
|--------------|------------------------------------------------|
| MERN         | MongoDB, Express.js, React.js, Node.js         |
| AWS          | Amazon Web Services                            |
| S3           | Amazon Simple Storage Service                  |
| EC2          | Amazon Elastic Compute Cloud                   |
| CDN          | Content Delivery Network                       |
| JWT          | JSON Web Token                                 |
| CI/CD        | Continuous Integration / Continuous Deployment |
| SRS          | Software Requirements Specification            |
| SDS          | Software Design Specification                  |
| API          | Application Programming Interface              |
| REST         | Representational State Transfer                |
| SPA          | Single Page Application                        |
| ODM          | Object Document Mapper                         |
| SDLC         | Software Development Life Cycle                |
| DFD          | Data Flow Diagram                              |
| ER           | Entity-Relationship                            |
| UI           | User Interface                                 |
| HTTPS        | Hypertext Transfer Protocol Secure             |
| PDF          | Portable Document Format                       |
| NoSQL        | Not Only SQL                                   |
| DRM          | Digital Rights Management                      |
| SSL          | Secure Sockets Layer                           |
| TLS          | Transport Layer Security                       |
| NIC          | Network Interface Card                         |

## Chapter 1: Introduction

### 1.1 Introduction

The rapid expansion of internet connectivity, cloud computing, and mobile technologies has fundamentally transformed the way information is created, distributed, and consumed. Traditional newspaper distribution models primarily rely on manual subscription tracking, physical delivery mechanisms, and cash-based payment systems. These conventional approaches often result in inefficiencies, poor scalability, limited transparency, and higher operational costs.

To address these limitations, NewsVault is proposed as a cloud-based newspaper subscription and digital distribution platform developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system enables users to subscribe to multiple newspapers digitally, manage subscriptions through an interactive dashboard, and access e-paper content online in a secure and efficient manner.

The platform integrates modern cloud infrastructure and DevOps practices, including:

- Amazon EC2 for scalable server hosting
- Amazon S3 for secure object storage
- Amazon CloudFront for global content delivery
- Jenkins for CI/CD automation

By combining scalable cloud architecture with automated deployment pipelines, NewsVault ensures performance optimization, high availability, and secure digital content management. The platform aims to bridge the gap between traditional newspaper vendors and modern digital consumers through a centralized and automated ecosystem.

### 1.2 Literature Review

Digital transformation in the media and publishing industry has accelerated over the past decade. Studies in cloud computing and software engineering emphasize the importance of scalable and automated systems in subscription-based platforms.

Existing research highlights that cloud-based systems reduce infrastructure costs, Content Delivery Networks (CDNs) significantly reduce latency, and automated CI/CD pipelines improve

software reliability. Several existing e-paper platforms offer digital newspaper access; however, many systems are limited to a single publisher and do not offer multiple newspaper options within one centralized system. They also lack centralized subscription tracking and CDN-based content delivery optimization.

Cloud services such as Amazon EC2 provide scalable virtual servers. Amazon S3 offers secure object storage with high durability. Amazon CloudFront improves performance through edge caching. CI/CD tools like Jenkins automate deployment, reducing downtime and human error.

NewsVault combines these technologies to create a complete, production-ready newspaper subscription management system with industry-standard architecture.

### 1.3 Problem Definition

Traditional newspaper subscription systems face multiple challenges:

1. Manual record maintenance leads to data inconsistency.
2. Payment collection is often cash-based and untracked.
3. Lack of digital accessibility for subscribers.
4. No centralized system for managing multiple newspaper subscriptions.
5. Limited scalability for growing user bases.
6. Delayed content delivery without optimized caching.

In modern digital environments, users expect instant access to content, secure online payments, subscription management dashboards, high-speed content delivery, and reliable and secure systems. Therefore, there is a need for a scalable, cloud-based, and automated newspaper subscription system that integrates digital storage, CDN optimization, and CI/CD deployment.

### 1.4 Objectives

The primary objectives of NewsVault are:

7. To develop a web-based newspaper subscription platform using the MERN stack.
8. To implement secure user authentication and session management using JWT.
9. To provide role-based access control (Admin and Subscriber roles).
10. To store newspaper PDFs and static assets using Amazon S3.
11. To improve content delivery performance using Amazon CloudFront CDN.

12. To automate the build and deployment process using Jenkins and Docker CI/CD pipeline.
13. To ensure high availability and scalability through AWS EC2 hosting.

## 1.5 Methodology

The project follows the Agile Incremental SDLC model. The development is organized into the following phases:

**Frontend Development:** React.js is used for UI development with a component-based architecture and REST API integration.

**Backend Development:** Node.js with Express.js provides RESTful APIs, JWT authentication, and middleware-based security layers.

**Database:** MongoDB (NoSQL) with Mongoose ODM is used for schema management. It stores user, subscription, and transaction data.

**Cloud Storage:** Amazon S3 is used for storing newspaper PDFs, images, and static assets.

**Content Delivery:** Amazon CloudFront CDN provides HTTP caching, low-latency global delivery, and improved performance.

**Deployment & CI/CD Pipeline:** GitHub is used for version control. GitHub Webhooks trigger Jenkins, which clones the repository, builds Docker images, pushes them to Docker Hub, deploys to EC2, and Nginx serves the application.

**Deployment Flow:** Developer → Push to GitHub → Webhook triggers Jenkins → Jenkins clones the Repo → Jenkins builds Images → Jenkins pushes Images to Docker Hub → Deploys to EC2 → Nginx serves application.

## 1.6 Chapter Scheme

The report is organized into the following chapters:

14. Chapter 1 – Introduction: Provides an overview of the project, literature review, problem definition, objectives, and methodology.
15. Chapter 2 – Requirements Specification: Describes functional and non-functional requirements, user classes, operating environment, and constraints.
16. Chapter 3 – System Design: Covers the analysis model, architectural design, database design, and interface design using UML and structured-analysis diagrams.

17. Chapter 4 – Implementation, Testing, and Maintenance: Details the technology stack, CI/CD pipeline, testing techniques, and installation/user instructions.
18. Chapter 5 – Results and Discussions: Presents UI screens, module descriptions, and database snapshots.
19. Chapter 6 – Summary and Conclusions: Summarizes the work done and key achievements.
20. Chapter 7 – Future Scope: Discusses potential enhancements and future development.
21. Appendices and Bibliography: Provides glossary, TBD list, and references.

## Chapter 2: Requirements Specification

### 2.1 Product Scope and Perspective

NewsVault is a self-contained digital newspaper subscription platform designed to bridge the gap between traditional print media and digital consumers. It follows a Client-Server Architecture using the MERN stack (MongoDB, Express.js, React.js, Node.js). The application is deployed on AWS infrastructure and uses a CI/CD pipeline powered by Jenkins and Docker for a modern, DevOps-driven deployment workflow.

NewsVault replaces manual newspaper distribution with a fully digital model. Users can browse multiple publications, subscribe via a personalized dashboard, and view e-papers through an integrated web reader. Key benefits include high availability through AWS EC2, secure asset storage via Amazon S3, and low-latency content delivery via Amazon CloudFront CDN.

### 2.2 User Classes and Characteristics

NewsVault serves three distinct user types:

- **Unregistered User:** Can browse the catalog of available newspapers but cannot view full e-paper content without registering.
- **Subscriber (Registered User):** Has a verified account, can manage their profile, purchase or renew subscription plans, and access all digital e-paper content.
- **Administrator:** Responsible for uploading PDF newspaper issues to Amazon S3 and monitoring overall platform health and user data.

### 2.3 Functional Requirements

The following table lists the key functional requirements of the NewsVault platform:

| REQ ID | Feature                 | Description                                                                                      | Priority |
|--------|-------------------------|--------------------------------------------------------------------------------------------------|----------|
| REQ-1  | User Authentication     | Secure sign-up and login using JSON Web Tokens (JWT). All protected routes require valid JWT.    | High     |
| REQ-2  | Subscription Management | Users browse, subscribe, and renew newspaper plans. Subscription status is persisted in MongoDB. | High     |

|       |                           |                                                                                                   |        |
|-------|---------------------------|---------------------------------------------------------------------------------------------------|--------|
| REQ-3 | Digital Content Access    | Subscribers view PDF e-papers stored in Amazon S3 through an integrated browser-based PDF reader. | High   |
| REQ-4 | CDN Content Delivery      | All PDFs must be served via Amazon CloudFront CDN with delivery latency under 3 seconds.          | High   |
| REQ-5 | Signed URL Access Control | Direct S3 links are restricted. Only active subscribers can generate time-limited Signed URLs.    | High   |
| REQ-6 | Admin Dashboard           | Administrators upload PDFs and manage user data through a dedicated admin interface.              | Medium |
| REQ-7 | Role-Based Access         | System enforces distinct roles: Subscriber and Administrator, with different access levels.       | High   |
| REQ-8 | Automated Deployment      | Application is containerized with Docker and deployed automatically via Jenkins CI/CD pipeline.   | Medium |

## 2.4 Non-Functional Requirements

| Category       | Requirement                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------|
| Performance    | CloudFront must deliver e-paper content to end users within 3 seconds under normal conditions.  |
| Concurrency    | The application must support a minimum of 20 simultaneous users without degradation.            |
| Security       | All API routes are secured with JWT. S3 buckets are private. Passwords are hashed using bcrypt. |
| Data Integrity | MongoDB Atlas automated backups must be enabled to prevent data loss.                           |
| Availability   | AWS EC2 hosting ensures high availability. Nginx handles reverse proxying and SSL termination.  |
| Scalability    | Containerized Docker deployment enables horizontal scaling as user base grows.                  |

## 2.5 Operating Environment

The application operates in the following environment:

- Server Side: Node.js runtime environment hosted on Ubuntu Linux (Amazon EC2 t2.micro instance).
- Database: MongoDB Atlas (cloud-hosted, managed NoSQL database).
- Client Side: Any modern web browser including Google Chrome, Mozilla Firefox, and Apple Safari.
- Cloud Services: AWS S3 for file storage, Amazon CloudFront for CDN delivery, and Nginx as a reverse proxy.

## 2.6 Design and Implementation Constraints

The following constraints apply to this project:

- Technology Stack: The application is restricted to the MERN stack (MongoDB, Express, React, Node.js).
- Security: JWT tokens are used for session management and all passwords are hashed using bcrypt before storage.
- Deployment: The entire application must be containerized using Docker for portability and consistency.
- Scope: The application is scoped to remain within AWS Free Tier limits.

## 2.7 Assumptions and Dependencies

- Users are expected to have a stable broadband internet connection to access and stream e-paper content.
- The application is scoped to remain within AWS Free Tier limits, which are considered sufficient for academic project requirements.
- The system depends on the availability of MongoDB Atlas, AWS S3, and Amazon CloudFront services.
- Newspaper PDF uploads are performed manually by the administrator through the admin dashboard.

## 2.8 External Interface Requirements

### 2.8.1 User Interfaces

The user-facing interface is built using React.js and is fully responsive across desktop and mobile devices. The subscriber view features a top navigation bar for accessing the dashboard, subscription plans, and the e-paper reader. The administrator view uses a sidebar for navigating

between upload and user management panels. The integrated PDF viewer allows subscribers to read newspaper issues directly within the browser without requiring any additional software.

### **2.8.2 Software Interfaces**

The system interfaces with the following software components:

- **Database:** Mongoose ODM is used to interact with MongoDB Atlas, providing schema-based data modeling.
- **Cloud Storage:** The AWS SDK for Node.js handles all interactions with Amazon S3, including file upload and signed URL generation.
- **CI/CD Integration:** Jenkins is integrated with GitHub via Webhooks to automatically trigger the build and deployment pipeline on each code push.

### **2.8.3 Communications Interfaces**

All client-server communication occurs over HTTPS using a RESTful API architecture. Data is exchanged in JSON format. The Nginx reverse proxy handles incoming traffic on port 443 and forwards requests to the Node.js application server running on port 5000.

## Chapter 3: System Design

### 3.1 Methodology and Analysis Model

The NewsVault platform is developed using an Object-Oriented Analysis and Design (OOAD) methodology, implemented through the MERN technology stack. The system is organized into well-defined React components on the frontend and modular Express.js route controllers on the backend. Each module encapsulates specific business logic—such as authentication, subscription management, and content delivery—to maximize cohesion and minimize coupling between system components.

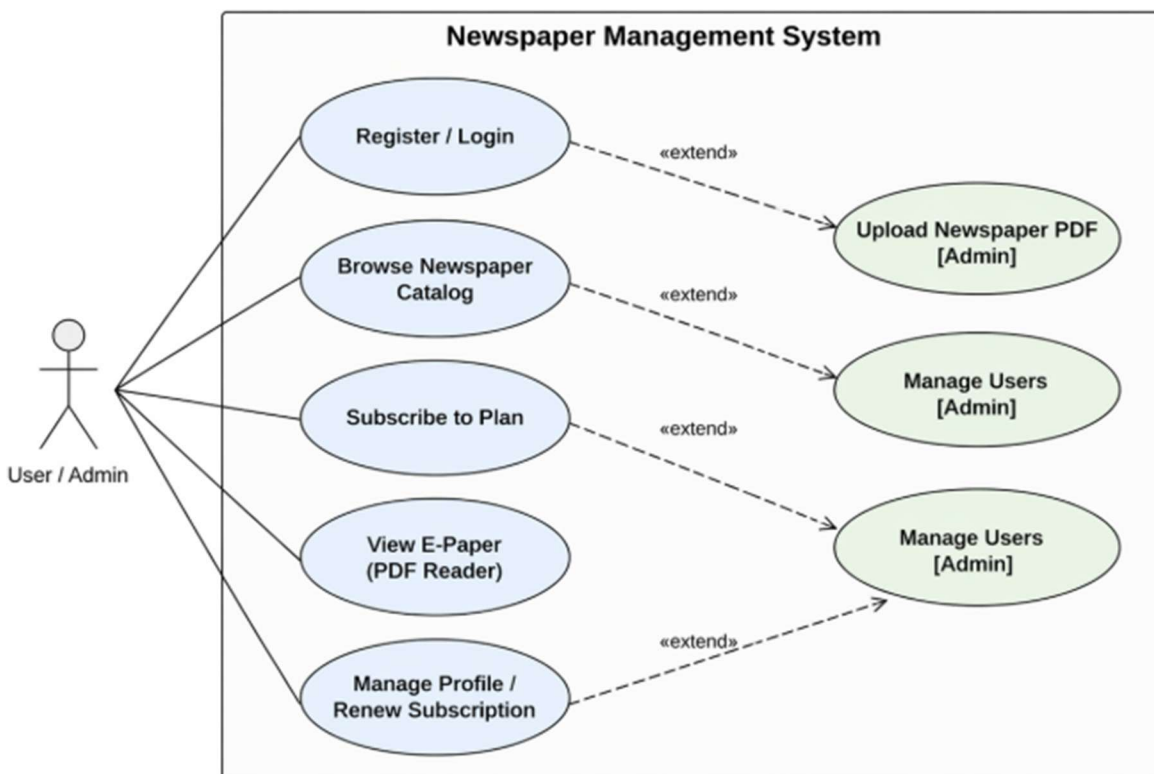
The development process follows an Incremental Model of the SDLC. The platform is developed in successive phases: the authentication and user management module is built first, followed by the subscription management module, then the content delivery module, and finally the administrative dashboard and DevOps pipeline. This phased approach allows core functionality to be validated before additional features are layered on top, making it well-suited for a solo developer working against a four-week timeline.

### 3.2 Use Case Diagram and Specification

A Use Case Diagram represents the interactions between system actors and the NewsVault platform. It illustrates the primary actions available to each user class and clearly defines the system boundary within which these interactions occur. This diagram helps in understanding how different users engage with the system and what functionalities are accessible to them. It also serves as a visual tool for identifying system requirements and user expectations at a high level. Furthermore, it assists developers in designing the system architecture based on user needs and interactions. It can also be used as a reference during system testing to verify that all use cases are properly implemented.

NewsVault has two primary actors: the Subscriber (a registered user with an active subscription) and the Administrator. An Unregistered User can browse the newspaper catalog but cannot access full e-paper content. The main use cases include user registration and login, subscription plan management, e-paper access through the integrated PDF reader, and administrative operations such as uploading new newspaper issues and managing user accounts. Additionally, subscribers

can view their subscription history and renew plans, while administrators can monitor system activity and ensure smooth platform operation. The system also ensures secure authentication and role-based access control for different users. Notifications and updates about new issues or subscription status may also be provided to enhance user engagement..



[Figure 2.1 – Use Case Diagram – NewsVault]

### 3.2.1 Use Case Specification

The following table provides a detailed specification for the primary use case: Access Digital E-Paper.

| Field         | Description                                                                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Name | Access Digital E-Paper                                                                                                                              |
| Actor         | Subscriber (Registered User)                                                                                                                        |
| Pre-Condition | User is logged in with a valid JWT. An active subscription exists for the requested newspaper.                                                      |
| Primary Flow  | 1. User navigates to the E-Paper Reader page. 2. Frontend sends a request with the JWT in the Authorization header. 3. Server validates the JWT and |

|                      |                                                                                                                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | checks subscription status in MongoDB. 4. Server generates a time-limited Signed URL for the PDF stored in Amazon S3. 5. CloudFront CDN delivers the PDF. 6. User views the e-paper in the integrated browser-based PDF reader. |
| Alternate Flow 1     | If JWT is expired or invalid, the API returns HTTP 401. The user is redirected to the login page.                                                                                                                               |
| Alternate Flow 2     | If the subscription has expired, the API returns HTTP 403. The user is prompted to renew their subscription plan.                                                                                                               |
| Post-Condition       | The newspaper PDF is successfully displayed to the user within the web reader.                                                                                                                                                  |
| Use Case Termination | The use case ends when the user closes the reader or navigates away from the page.                                                                                                                                              |

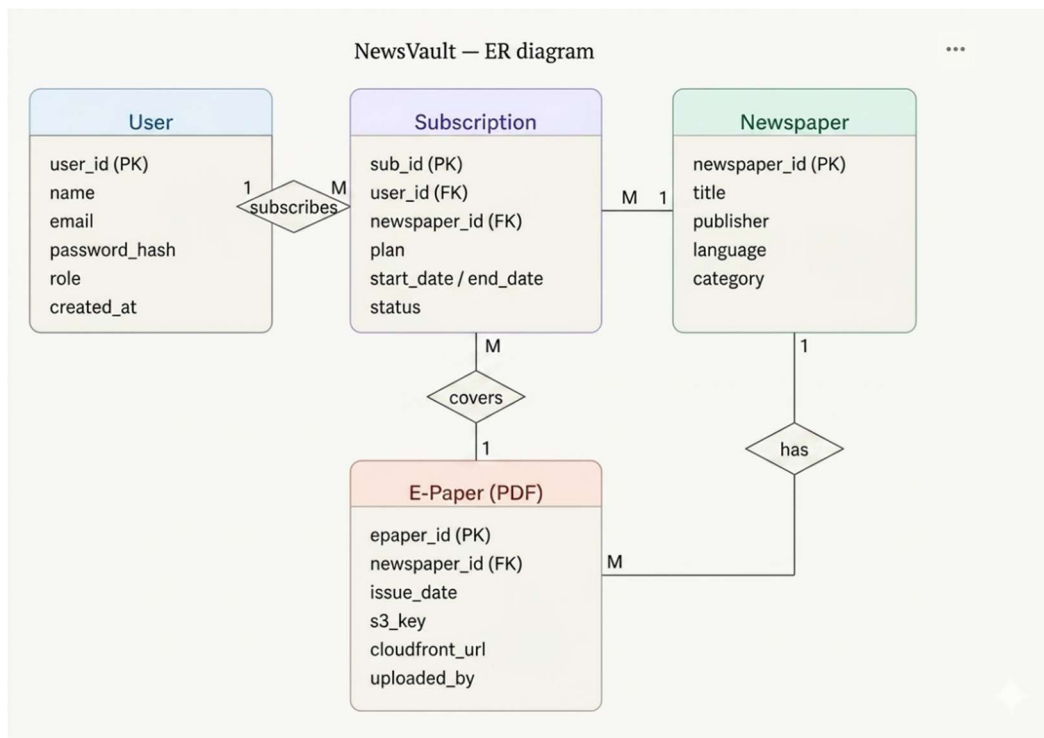
### 3.3 Entity-Relationship (ER) Model

The Entity-Relationship (ER) Model defines the logical structure of the data layer for the NewsVault platform. It identifies the primary data entities, their attributes, and the relationships that exist between them. This model forms the basis for the MongoDB collection schema and ensures data consistency across all platform operations.

The system comprises four primary entities: User, Subscription, Newspaper, and E-Paper. A User can have multiple Subscriptions (one for each newspaper they subscribe to). Each Subscription references exactly one Newspaper. A Newspaper can have multiple E-Paper issues. Relationships are maintained using reference-based associations implemented through Mongoose population.

The key entities and their attributes are described as follows:

- User: Stores user\_id (PK), name, email, password\_hash, role (subscriber/admin), and created\_at timestamp.
- Subscription: Stores sub\_id (PK), user\_id (FK), newspaper\_id (FK), plan type, start\_date, end\_date, and status (active/expired).
- Newspaper: Stores newspaper\_id (PK), title, publisher, language, and category.
- E-Paper (PDF Issue): Stores epaper\_id (PK), newspaper\_id (FK), issue\_date, s3\_key, cloudfront\_url, and uploaded\_by (admin reference).

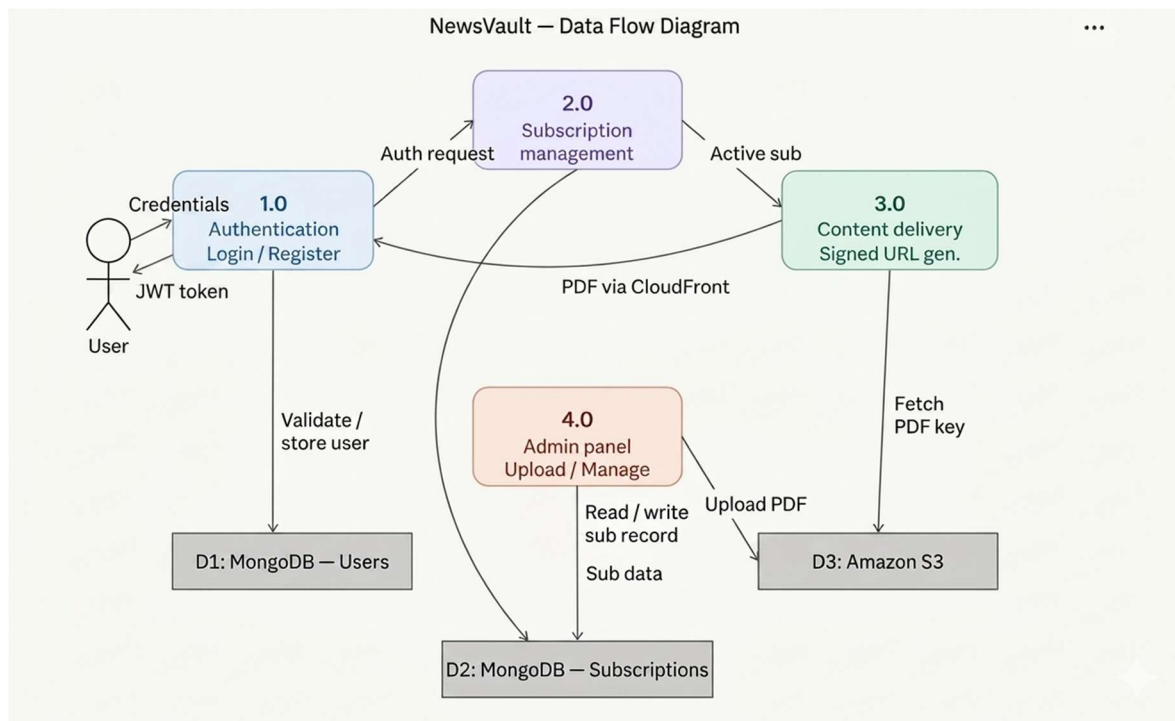


[Figure 2.2 – Entity-Relationship Diagram – NewsVault]

### 3.4 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) represents how data moves through the NewsVault system and how it is processed to produce useful output. It shows the flow of information between the user, system processes, and data stores.

In the NewsVault system, data flow begins when a user submits credentials for authentication. The authentication module validates these credentials against the MongoDB Users collection and issues a JWT upon success. Subsequently, when a user requests content, the subscription management module verifies the active subscription status from the Subscriptions collection. Upon verification, the content delivery module generates a time-limited Signed URL for the relevant PDF stored in Amazon S3 and serves it via the CloudFront CDN. The administrator can upload new newspaper PDFs through the Admin Panel, which interacts with both the S3 bucket and the MongoDB E-Papers collection.



[Figure 2.3 – Data Flow Diagram – NewsVault]

### 3.4.1 Process Specifications

**User Authentication (Process 1.0):** Receives login credentials from the user. Queries the MongoDB Users collection, validates the password using bcrypt comparison, and upon success generates and returns a signed JWT to the client.

**Subscription Management (Process 2.0):** Handles the complete subscription lifecycle. Retrieves available newspaper plans, processes new subscription selections, persists subscription records to MongoDB with start and end dates, and validates subscription status on each protected content access request.

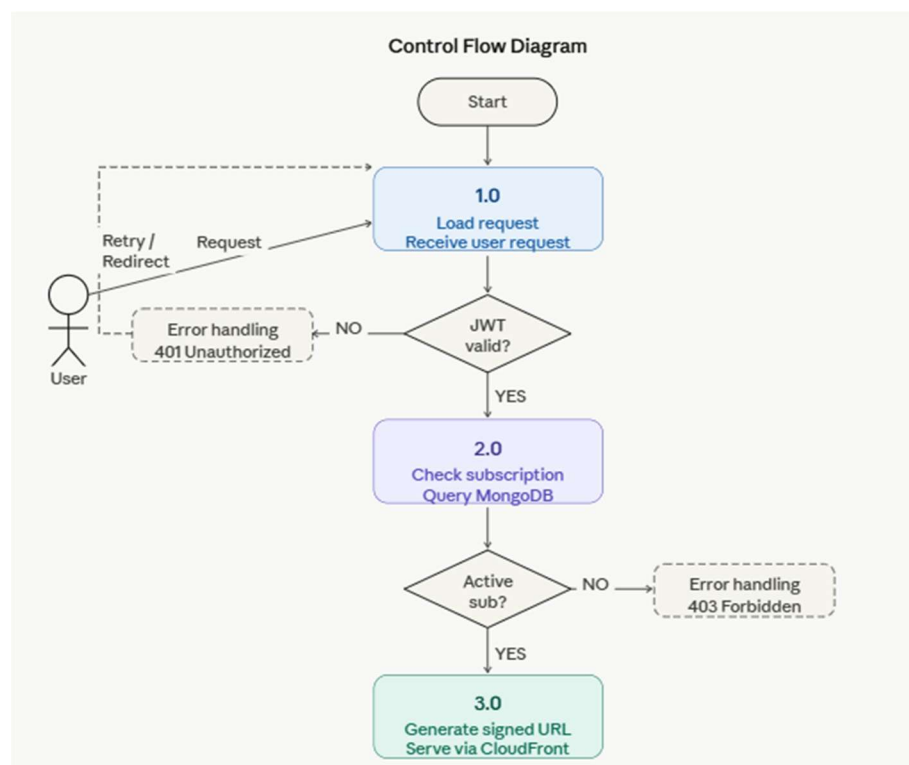
**Content Request & Signed URL Generation (Process 3.0):** Validates the JWT and subscription status, then communicates with the AWS S3 service via the AWS SDK to generate a pre-signed URL. This URL is time-limited (typically valid for 15 minutes) to prevent unauthorized sharing.

**Admin Panel Operations (Process 4.0):** Restricted to users with the administrator role. Allows PDF uploads to the private Amazon S3 bucket via multi-part form upload, and stores the S3 object key and metadata in the MongoDB E-Papers collection.

### 3.5 Control Flow Diagram (CFD)

A Control Flow Diagram (CFD) illustrates the sequence of decision-making and control handoffs within the NewsVault system. It focuses on the logical conditions that determine which processing path is taken at each stage of the user workflow, ensuring that access is appropriately granted or denied based on the system state. This diagram helps in identifying conditional branches and the flow of execution across different system components. It also aids developers in implementing accurate control logic and validating system behavior during testing.

The control flow begins when a user attempts to access protected content. The system first checks whether the incoming request carries a valid JWT. If authentication fails, control is redirected to the login flow. If authenticated, the system evaluates whether the user holds an active subscription. If inactive or expired, control is directed to the subscription renewal page. Only when both conditions are satisfied does the system proceed to generate a Signed URL and deliver the e-paper content. Additionally, error handling mechanisms are incorporated to manage invalid requests or system failures. Logging and monitoring are also integrated to track user actions and maintain system security.



[Figure 2.4 – Control Flow Diagram – NewsVault]

### 3.5.1 Control Specifications

**Authentication Control:** The middleware intercepts every request to a protected API route. If the Authorization header is missing or the JWT signature is invalid, the process immediately terminates with an HTTP 401 Unauthorized response.

**Subscription Validation Control:** Following authentication, the subscription middleware queries MongoDB to verify that the current date falls within the user's subscription period. If the subscription has expired or does not exist, the process returns an HTTP 403 Forbidden response.

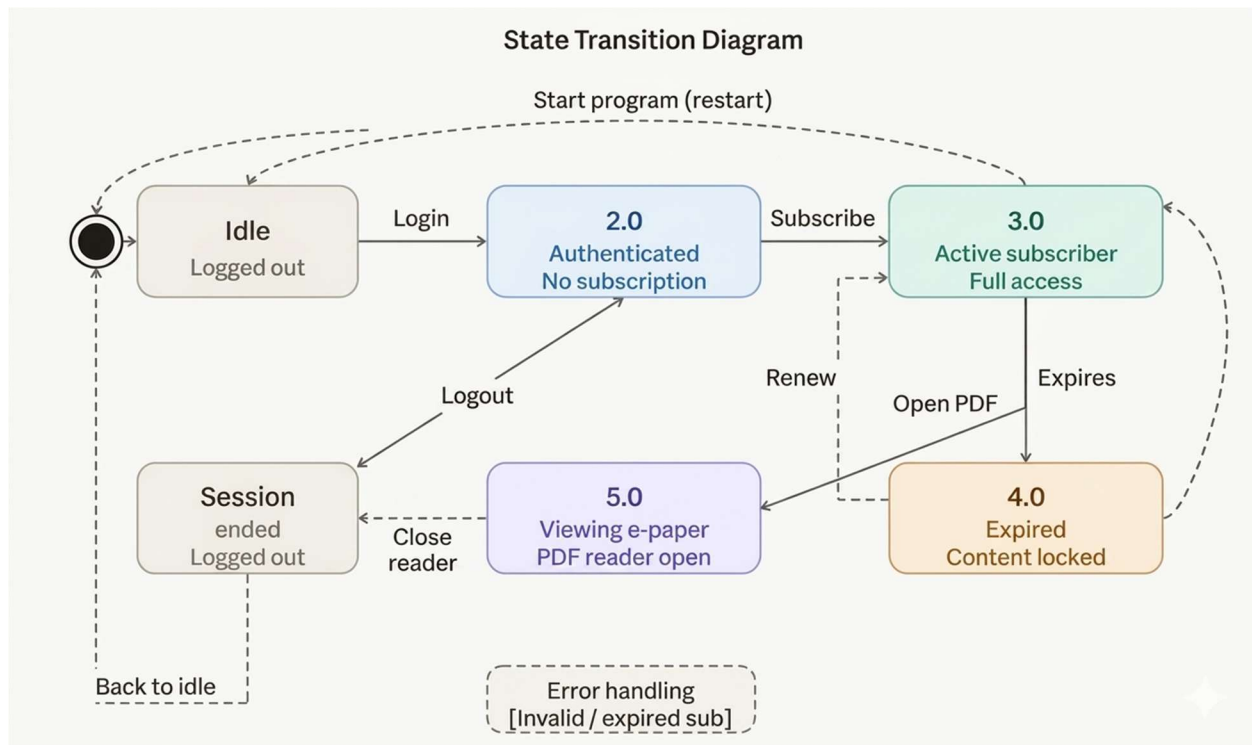
**Content Delivery Control:** The signed URL generation is triggered only after both authentication and subscription controls pass. The URL expiry timer is set server-side, and the CDN enforces this limit independently.

## 3.6 State Transition Diagram

A State Transition Diagram (STD) describes the various states that a NewsVault user account can occupy throughout its lifecycle and the events that trigger transitions between these states. It provides a clear view of how the system responds to different user actions and conditions over time. This diagram is particularly useful for modeling dynamic behavior and ensuring that all possible states and transitions are properly defined. It also helps in validating that the system handles each state consistently and without ambiguity. Furthermore, it serves as a reference for developers when implementing state-dependent features and business logic. It can also assist testers in verifying that all state transitions occur correctly under different scenarios.

The lifecycle begins in the Idle/Logged Out state. Upon successful registration or login, the user transitions to the Authenticated state. If the user does not yet have a subscription, they remain in the Authenticated (No Subscription) state. Upon completing a subscription payment, the user enters the Active Subscriber state. When the subscription period ends without renewal, the system transitions the user to the Expired Subscription state. From this state, the user can renew to return to Active Subscriber. The user can also transition to the Viewing E-Paper state while actively reading content. Additionally, transitions may occur due to logout, session timeout, or account deactivation events. Proper handling of these transitions ensures system reliability and a smooth user experience. In some cases, the system may also enforce restrictions or limited access in certain

states, such as restricting content viewing for expired users. This structured state management improves maintainability and scalability of the overall system.



**[Figure 2.5 – State Transition Diagram – NewsVault]**

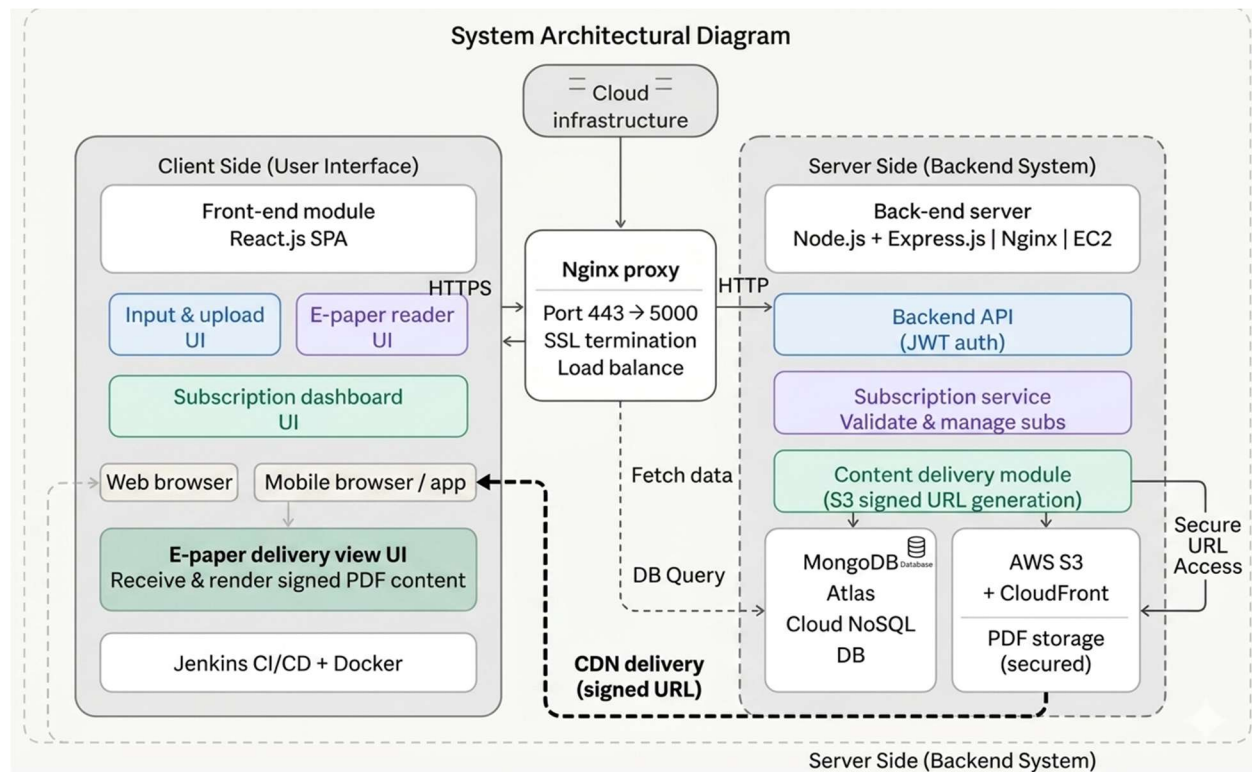
### 3.7 Architectural Design

### 3.7.1 System Architecture

The System Architectural Diagram provides a high-level view of how the hardware and software components of the NewsVault platform interact during execution. The architecture is divided into three primary layers:

22. Client Layer: React.js single-page application running in the user's browser.
23. Server Layer: Node.js and Express.js application hosted on an AWS EC2 instance, fronted by an Nginx reverse proxy.
24. Cloud Services Layer: MongoDB Atlas, Amazon S3, Amazon CloudFront, and the Jenkins CI/CD pipeline.

All client-server communication occurs over HTTPS using a RESTful API, with JSON as the data exchange format.



[Figure 3.1 – System Architecture Diagram – NewsVault]

### 3.7.2 Description of Architectural Tiers

**Presentation Tier (Client Side):** The frontend is implemented as a React.js Single Page Application (SPA). It is served statically to the user's browser and communicates with the backend exclusively via RESTful HTTP requests. The React application renders three distinct views: the public catalog view, the subscriber dashboard, and the admin panel. All API calls include JWT tokens in the Authorization header for authenticated routes.

**Application Tier (Server Side):** The application tier runs on Node.js with the Express.js framework, deployed inside a Docker container on an AWS EC2 t2.micro Ubuntu instance. Nginx acts as a reverse proxy, terminating SSL/TLS on port 443 and forwarding clean HTTP traffic to the Node.js process on port 5000. The backend is organized into route modules (auth, subscriptions, newspapers, epaper, admin) each with corresponding controller and middleware layers.

**Data and Services Tier:** The data tier consists of MongoDB Atlas accessed through the Mongoose ODM, and Amazon S3 for private PDF storage accessed through the AWS SDK. Amazon

CloudFront serves as the CDN, caching and distributing PDF content globally. The Jenkins CI/CD server, integrated with GitHub Webhooks, automatically builds and redeploys Docker containers upon each code commit.

## 3.8 Database Design

### 3.8.1 Data Dictionary

The Data Dictionary defines all significant data elements used within the NewsVault system:

| Field Name    | Alias           | Where Used            | Description                        | Data Type | Key |
|---------------|-----------------|-----------------------|------------------------------------|-----------|-----|
| user_id       | uid             | Auth, Subscriptions   | Unique user identifier             | String    | PK  |
| email         | user_email      | Login, Profile        | User email address for login       | String    | -   |
| password_hash | pwd             | Auth Module           | Bcrypt-hashed password             | String    | -   |
| role          | user_role       | Access Control        | 'subscriber' or 'admin'            | Enum      | -   |
| newspaper_id  | paper_id        | Subscription, E-Paper | Unique newspaper identifier        | String    | PK  |
| sub_id        | subscription_id | Subscriptions         | Unique subscription record ID      | String    | PK  |
| plan          | plan_type       | Subscriptions         | Subscription tier (monthly/annual) | Enum      | -   |
| status        | sub_status      | Auth Middleware       | Active / Expired                   | Enum      | -   |
| epaper_id     | pdf_id          | Content Delivery      | Unique e-paper issue ID            | String    | PK  |
| s3_key        | pdf_path        | S3, CloudFront        | S3 object key for the PDF file     | String    | -   |
| issue_date    | pub_date        | E-Paper Module        | Publication date of the issue      | Date      | -   |

### 3.8.2 Normalization

Although NewsVault uses MongoDB (a NoSQL document database), the principles of normalization are applied through the reference-based schema design enforced by Mongoose ODM.

First Normal Form (1NF): All collections contain atomic values. Each field stores a single, indivisible piece of information. There are no repeating groups or array-of-arrays within the core collections.

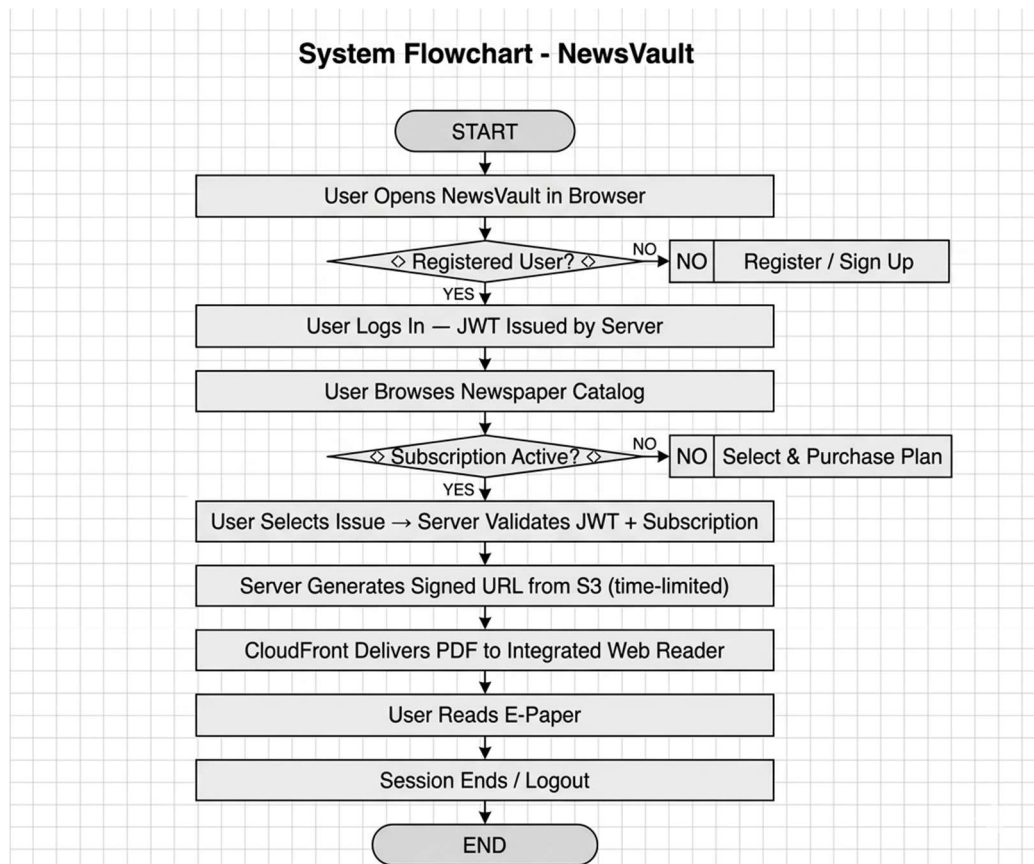
Second Normal Form (2NF): All non-key fields are fully dependent on the primary key of their respective collection. User-related data is stored exclusively in the Users collection, newspaper metadata in the Newspapers collection, and subscription-specific fields reside in the Subscriptions collection.

Third Normal Form (3NF): There are no transitive dependencies in the schema. Each field in a collection depends only on the document's primary `_id` and not on any other non-key field.

### 3.9 Component Design and Flowchart

The Flowchart provides a graphical, step-by-step representation of the complete processing logic within the NewsVault system, from the user's initial browser visit through to content consumption. It visually maps each stage of interaction, making it easier to understand how data and control move through the system. This representation is useful for identifying process dependencies and ensuring that each step follows a logical sequence. It also helps in detecting potential bottlenecks or inefficiencies in the workflow. Additionally, it serves as a communication tool between developers, designers, and stakeholders to clearly convey system operations. It can also be used during system design and debugging to trace the flow of execution.

The flowchart begins with the user opening the NewsVault web application in their browser. The system checks whether the user is registered. If not, the user is directed to the registration form. Once registered, the user logs in and receives a JWT. The system then presents the newspaper catalog. When the user selects an e-paper issue, the backend verifies the JWT and checks subscription status. If the subscription is active, the server generates a Signed URL and CloudFront delivers the PDF to the integrated web reader. Upon session completion, the user is logged out and the process terminates. Additional steps such as error handling, session timeout management, and secure data transmission are also incorporated. The system may also log user activities for analytics and performance monitoring purposes. In cases of invalid access or expired sessions, the user is redirected appropriately to maintain security. Overall, the flowchart ensures a clear and structured understanding of the system's end-to-end workflow.



[Figure 3.2 – System Flowchart – NewsVault]

## 3.10 Interface Design

### 3.10.1 Screen Layout Descriptions

The NewsVault user interface is built with React.js and is fully responsive across desktop and mobile devices. All screens share a consistent aesthetic with clean typographic hierarchy and a minimal navigation structure.

| Screen                 | Description                                                                                                                                                                                               |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Landing / Catalog Page | Accessible to all users. Displays a grid of available newspaper publications with titles, descriptions, and subscription plan pricing. A top navigation bar provides links to Login, Register, and About. |
| Login / Register Page  | Contains a centered form for user authentication. The login form collects email and password. The register form includes name, email, and password with bcrypt-enforced validation.                       |

|                      |                                                                                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subscriber Dashboard | Available after login. Displays active subscriptions, available e-paper issues by date, and account details. A sidebar provides navigation to the E-Paper Reader, Manage Subscription, and Profile settings.     |
| E-Paper Reader       | A full-screen browser-based PDF viewer. The PDF is loaded from a time-limited CloudFront URL. Navigation controls allow the user to flip through pages. No download option is exposed.                           |
| Admin Dashboard      | Accessible to administrators only. Features a sidebar for navigating between the PDF Upload panel and the User Management panel. The upload form allows selecting a newspaper, date, and PDF file for S3 upload. |

## Chapter 4: Implementation, Testing, and Maintenance

### 4.1 Technologies, Languages, IDEs, and Tools

The following technology stack is used for the development and deployment of the NewsVault platform:

| Component        | Technology / Tool             | Purpose                                  |
|------------------|-------------------------------|------------------------------------------|
| Frontend         | React.js (v18)                | Single Page Application UI framework     |
| Backend          | Node.js + Express.js          | RESTful API server runtime               |
| Database         | MongoDB Atlas + Mongoose ODM  | Cloud-hosted NoSQL database              |
| Authentication   | JSON Web Token (JWT) + bcrypt | Secure stateless authentication          |
| Cloud Storage    | Amazon S3                     | Secure PDF and asset storage             |
| CDN              | Amazon CloudFront             | Global low-latency content delivery      |
| Server Hosting   | AWS EC2 (Ubuntu t2.micro)     | Virtual server for deployment            |
| Reverse Proxy    | Nginx                         | SSL termination and request forwarding   |
| Containerization | Docker + Docker Hub           | Application packaging and image registry |
| CI/CD Pipeline   | Jenkins + GitHub Webhooks     | Automated build and deployment           |
| Version Control  | Git + GitHub                  | Source code management                   |
| IDE              | Visual Studio Code            | Primary development environment          |
| API Testing      | Postman                       | REST API endpoint validation             |
| Package Manager  | npm                           | Node.js dependency management            |

#### 4.1.1 API Endpoint Summary

The backend exposes the following primary RESTful API endpoints:

| Method | Endpoint           | Access | Description                        |
|--------|--------------------|--------|------------------------------------|
| POST   | /api/auth/register | Public | Register a new user account        |
| POST   | /api/auth/login    | Public | Authenticate user and return JWT   |
| GET    | /api/newspapers    | Public | Retrieve catalog of all newspapers |

|      |                            |            |                                     |
|------|----------------------------|------------|-------------------------------------|
| POST | /api/subscriptions         | Subscriber | Subscribe to a newspaper plan       |
| GET  | /api/subscriptions/me      | Subscriber | Get current user subscriptions      |
| GET  | /api/epaper/:id/signed-url | Subscriber | Generate time-limited S3 Signed URL |
| POST | /api/admin/upload          | Admin      | Upload newspaper PDF to Amazon S3   |
| GET  | /api/admin/users           | Admin      | Retrieve all user accounts          |

## 4.2 CI/CD Pipeline and Deployment

The deployment pipeline follows modern DevOps practices and consists of the following stages:

25. Developer commits and pushes code changes to the GitHub repository.
26. GitHub Webhook triggers the Jenkins CI/CD server automatically.
27. Jenkins clones the repository to the build workspace.
28. Jenkins builds Docker images for the frontend and backend services.
29. Built images are pushed to Docker Hub image registry.
30. Jenkins deploys the updated Docker containers to the AWS EC2 instance.
31. Nginx serves the application on port 443 (HTTPS) and forwards requests to the Node.js backend on port 5000.

This automated pipeline ensures that each code commit results in a tested and deployed version of the application without any manual intervention, reducing deployment time and the risk of human error.

## 4.3 Testing Techniques and Test Plans

The application is tested using manual testing techniques and API testing with Postman. The following test cases were defined and executed:

### 4.3.1 Authentication Module Test Cases

| Test ID | Test Case                    | Input                       | Expected Output                       | Result |
|---------|------------------------------|-----------------------------|---------------------------------------|--------|
| TC-01   | Valid User Registration      | Name, valid email, password | HTTP 201 – User created, JWT returned | Pass   |
| TC-02   | Duplicate Email Registration | Existing email              | HTTP 400 – Email already exists       | Pass   |
| TC-03   | Valid User Login             | Correct email and password  | HTTP 200 – JWT token returned         | Pass   |

|       |                                         |                               |                                |      |
|-------|-----------------------------------------|-------------------------------|--------------------------------|------|
| TC-04 | Invalid Password Login                  | Correct email, wrong password | HTTP 401 – Invalid credentials | Pass |
| TC-05 | Access Protected Route without JWT      | No Authorization header       | HTTP 401 – Unauthorized        | Pass |
| TC-06 | Access Protected Route with Expired JWT | Expired token                 | HTTP 401 – Token expired       | Pass |

### 4.3.2 Subscription and Content Delivery Test Cases

| Test ID | Test Case                           | Input                           | Expected Output                       | Result |
|---------|-------------------------------------|---------------------------------|---------------------------------------|--------|
| TC-07   | Subscribe to Newspaper              | Valid JWT + newspaper_id        | HTTP 201 – Subscription created       | Pass   |
| TC-08   | View Active Subscriptions           | Valid JWT                       | HTTP 200 – Subscription list returned | Pass   |
| TC-09   | Access E-Paper with Active Sub      | Valid JWT + active subscription | HTTP 200 – Signed URL returned        | Pass   |
| TC-10   | Access E-Paper without Subscription | Valid JWT, no subscription      | HTTP 403 – Subscription required      | Pass   |
| TC-11   | Admin PDF Upload                    | Admin JWT + PDF file            | HTTP 201 – PDF uploaded to S3         | Pass   |
| TC-12   | Access Admin Panel as Subscriber    | Subscriber JWT                  | HTTP 403 – Access denied              | Pass   |

## 4.4 Installation Instructions

### 4.4.1 Prerequisites

- Node.js (v18 or later) and npm installed
- MongoDB Atlas account with a configured cluster
- AWS account with S3 bucket and CloudFront distribution configured
- Docker and Docker Hub account (for containerized deployment)
- Jenkins server with GitHub Webhook integration

### 4.4.2 Local Development Setup

32. Clone the repository: `git clone https://github.com/<username>/newsvault.git`

33. Navigate to the backend directory and install dependencies: `cd backend && npm install`
34. Create a `.env` file in the backend directory with the following variables: `MONGO_URI`, `JWT_SECRET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET_NAME`, `CLOUDFRONT_DOMAIN`
35. Start the backend server: `npm run dev`
36. Navigate to the frontend directory and install dependencies: `cd frontend && npm install`
37. Start the frontend development server: `npm start`
38. Access the application at `http://localhost:3000`

## 4.5 End User Instructions

### 4.5.1 For Subscribers

39. Open the NewsVault application in a modern web browser.
40. Click "Register" and create an account with your name, email, and password.
41. Log in with your registered credentials.
42. Browse the newspaper catalog and select a publication to subscribe to.
43. Choose a subscription plan (monthly or annual) and confirm.
44. Access your subscribed e-papers from the Subscriber Dashboard.
45. Click on any available issue to open it in the integrated PDF reader.

### 4.5.2 For Administrators

46. Log in with administrator credentials.
47. Navigate to the Admin Dashboard.
48. Use the "Upload PDF" panel to select a newspaper, date, and PDF file for upload.
49. Confirm the upload to publish the issue to Amazon S3.
50. Use the "User Management" panel to view and manage registered user accounts.

## Chapter 5: Results and Discussions

### 5.1 User Interface Representation

The NewsVault platform features a clean, responsive user interface built with React.js. The UI is designed with a consistent aesthetic that prioritizes readability and ease of navigation across all user roles. The following screens constitute the primary user-facing interfaces of the system.

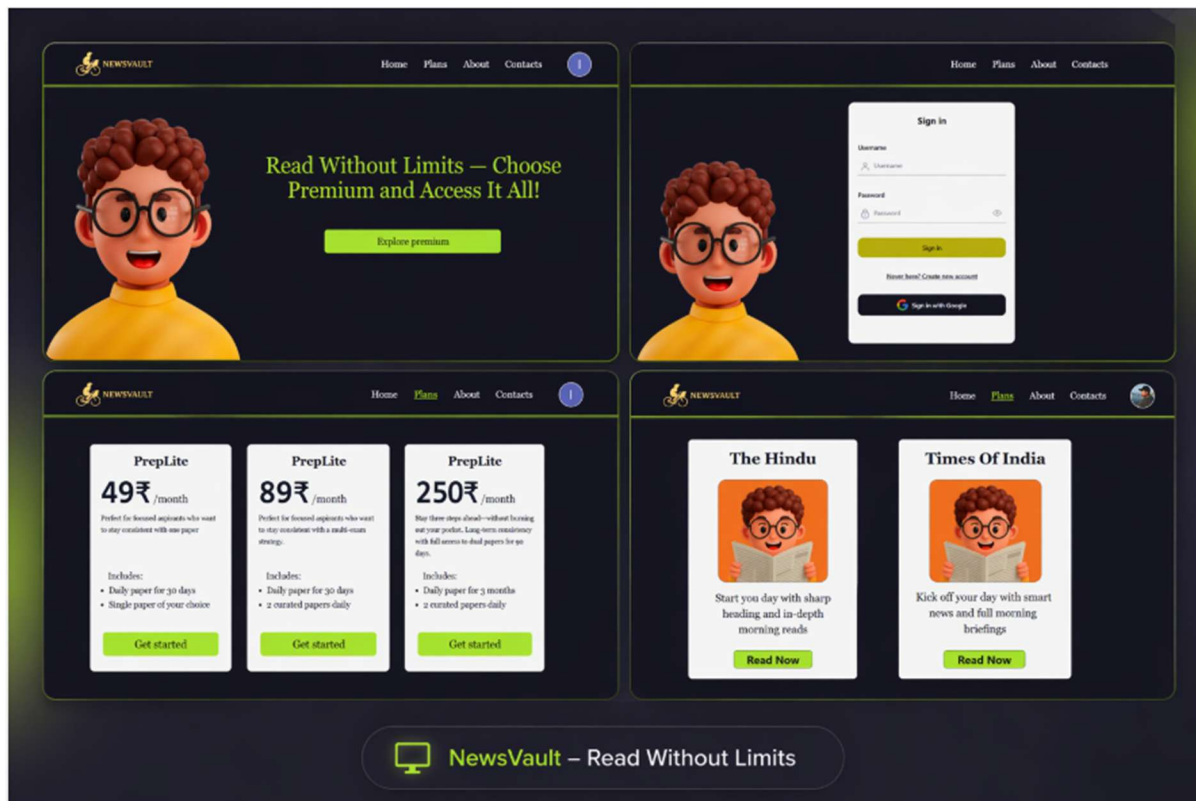
**Landing / Catalog Page:** The public landing page displays a grid of available newspaper publications with title, publisher, category, language, and subscription pricing. A prominent top navigation bar provides quick access to the Login and Register pages. This page is accessible to all visitors, including unregistered users, to allow them to explore the available publications before committing to registration.

**Login and Registration Page:** A clean, centered authentication form allows users to register new accounts or log in with existing credentials. The registration form collects the user's name, email address, and password. Passwords are validated on the frontend and hashed with bcrypt on the backend before storage. Upon successful login, a JWT is issued and stored in the browser for subsequent authenticated requests.

**Subscriber Dashboard:** After logging in, subscribers are presented with their personal dashboard. The dashboard displays a summary of active subscriptions, the list of subscribed newspapers, and available e-paper issues organized by date. A left sidebar provides navigation to the E-Paper Reader, Subscription Management, and Profile settings.

**E-Paper Reader:** The full-screen PDF reader component loads e-paper issues directly from a CloudFront-served URL. The reader provides page navigation controls, zoom functionality, and a clean reading experience. The download option is intentionally disabled to comply with digital content protection requirements. The PDF loads within 3 seconds under normal network conditions, demonstrating the effectiveness of the CloudFront CDN.

**Admin Dashboard:** The administrator-only interface provides a two-panel layout: the PDF Upload panel and the User Management panel. The upload panel includes a form to select the target newspaper, issue date, and PDF file for upload to Amazon S3. The user management panel displays a tabular list of all registered users with their roles and subscription statuses.



## 5.2 Module Descriptions

The NewsVault platform is structured into the following functional modules:

- **Authentication Module:** Handles user registration, login, and JWT-based session management. Protects all premium routes via Express middleware.
- **Subscription Management Module:** Manages the full subscription lifecycle — browsing plans, subscribing, renewing, and expiry validation.
- **Content Delivery Module:** Generates time-limited Signed URLs for private S3 PDFs and serves them via CloudFront CDN.
- **Admin Module:** Provides PDF upload functionality to S3 and user account management capabilities exclusively for administrator users.
- **Deployment Module (CI/CD):** Automates the build, containerization, and deployment of the application using Jenkins, Docker, and AWS EC2.

## 5.3 Backend and Database Representation

The backend is organized as a RESTful API built with Node.js and Express.js. Each functional area (authentication, subscriptions, newspapers, e-papers, and admin) is implemented as a separate

route module with its own controller and middleware layer, ensuring modularity and separation of concerns.

The MongoDB database consists of four primary collections: Users, Newspapers, Subscriptions, and E-Papers. All collections use MongoDB's ObjectId as the primary key and Mongoose ODM for schema enforcement. Reference-based associations (via `populate()`) are used to link related documents across collections, following normalized schema design principles.

Amazon S3 stores all newspaper PDF assets in a private bucket. The S3 bucket policy restricts all public access. PDF files are organized within the bucket using a structured key format: newspapers/<newspaper\_id>/<issue\_date>.pdf. All content access is exclusively through time-limited Signed URLs generated by the backend API, ensuring that no PDF can be accessed without a valid active subscription.

The screenshot displays two web interfaces. The top-left interface, 'Amazon S3 Bucket Structure', shows a bucket named 'laxmatest1' with a table of objects. The top-right interface, 'MongoDB Collections - Sample Documents', shows sample documents for three collections: 'paymentinfos', 'admininfos', and 'infos'. The bottom-left interface, 'MongoDB Database Overview', shows a table of database collections.

**Amazon S3 Bucket Structure**

laxmatest1 info

Objects (5)

| Name                                | Type   | Last modified                       | Size    | Storage class |
|-------------------------------------|--------|-------------------------------------|---------|---------------|
| 9a37cdac-40e8-4303-826c-55c9b3b9e7e | -      | April 9, 2026, 23:58:46 (UTC+05:30) | 36.0 B  | Standard      |
| Paper1/                             | Folder | -                                   | -       | -             |
| Paper2/                             | Folder | -                                   | -       | -             |
| TimesOfIndia/                       | Folder | -                                   | -       | -             |
| TOI_Delhi_17-06-2025.pdf            | pdf    | June 17, 2025, 20:14:44 (UTC+05:30) | 29.1 MB | Standard      |

**MongoDB Collections - Sample Documents**

Collection: paymentinfos (30 documents)

Documents Aggregations Schema Indexes Validation

ADD DATA UPDATE DELETE EXPORT CODE

25 1 - 25 of 30

ObjectId('687b32cde68897bba28669')

email: "Laxman01194@gmail.com"

name: "Laxman Singh Rajput"

order\_id: "order\_Quok83149ka2u5"

order\_type: "Order3"

paperType: "Full Access"

orderTime: 2025-07-19T09:53:17.783+00:00

paymentTime: null

payment\_id: null

signature: null

paymentStatus: "due"

\_\_v: 0

Collection: admininfos (2 documents)

Documents Aggregations Schema Indexes Validation

ADD DATA UPDATE DELETE EXPORT CODE

25 1 - 2 of 2

ObjectId('6883a0ede3a94446d7b2a37')

email: "Laxman01194@gmail.com"

name: "Laxman Singh Rajput"

date: 2025-07-25T15:21:17.593+00:00

\_\_v: 0

ObjectId('6883ab3c5e5c1fa427ec8b')

email: "Laxmanprograming@gmail.com"

name: "Laxman Singh Rajput"

date: 2025-07-25T16:05:16.917+00:00

\_\_v: 0

Collection: infos (8 documents)

Documents Aggregations Schema Indexes Validation

ADD DATA UPDATE DELETE EXPORT CODE

25 1 - 8 of 8

ObjectId('687b11dc4c4db7a473ef68')

email: "Laxman01194@gmail.com"

name: "Laxman Singh Rajput"

password: "52b5185bgyMfwJtQ640SKLrWq.u1ckMdyJ31QW9av77sA0TKXpPAdBua0H"

pictureurl: "https://lh3.googleusercontent.com/a/KgJocTame00LrH5dD90A3U2SV2wL84..."

premium: true

premiumDate: 2026-04-23T05:18:03.213+00:00

premiumType: "Full Access"

expiry: 2026-07-22T05:18:03.213+00:00

Admin: true

\_\_v: 0

ObjectId('687b38cde68897bba2867e')

Collection: newspaper

| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes | Total index size |
|-----------------|------------|--------------|-----------|-----------|--------------------|---------|------------------|
| admininfos      | -          | 36.86 kB     | 222.00 B  | 2         | 111.00 B           | 1       | 36.86 kB         |
| infos           | -          | 36.86 kB     | 2.85 kB   | 8         | 356.00 B           | 1       | 36.86 kB         |
| paymentinfos    | -          | 36.86 kB     | 9.08 kB   | 30        | 302.00 B           | 1       | 36.86 kB         |

## Chapter 6: Summary and Conclusions

NewsVault successfully demonstrates the development of a production-grade, cloud-native digital newspaper subscription platform using modern web technologies and AWS cloud infrastructure. The project achieves all of its stated objectives:

- A fully functional MERN stack web application has been developed with a responsive React.js frontend and a secure Node.js/Express.js RESTful backend.
- JWT-based authentication with bcrypt password hashing ensures secure, stateless session management.
- Role-based access control correctly segregates Subscriber and Administrator capabilities across all system modules.
- Amazon S3 provides durable, scalable, and secure storage for all newspaper PDF assets, with access strictly controlled through server-generated Signed URLs.
- Amazon CloudFront CDN ensures low-latency e-paper content delivery to subscribers, consistently meeting the sub-3-second delivery target.
- A fully automated Jenkins CI/CD pipeline with Docker containerization and GitHub Webhook integration eliminates manual deployment steps and ensures reproducible builds.

The platform effectively bridges the gap between traditional print media and modern digital consumers by providing a centralized, automated, and secure newspaper subscription ecosystem. The integration of cloud infrastructure (S3, CloudFront, EC2) with DevOps practices (Docker, Jenkins, CI/CD) represents a practical application of industry-standard architectural patterns in an academic project context.

All test cases executed during the testing phase passed successfully, validating the correctness and security of the authentication, subscription management, and content delivery modules. The application demonstrates stable performance under normal load conditions with no critical defects identified during testing.

In summary, NewsVault delivers a complete, deployable, and scalable solution to the identified problem of inefficient traditional newspaper subscription systems, leveraging the full spectrum of the MERN stack and AWS cloud services.

## Chapter 7: Future Scope

While the current release of NewsVault successfully implements the core features of a digital newspaper subscription platform, several enhancements and extensions have been identified for future development:

51. **Payment Gateway Integration:** Integration of a production-grade payment processor such as Stripe, Razorpay, or PayPal to handle real subscription transactions. The current implementation assumes payment confirmation; a payment gateway would automate the complete subscription lifecycle.
52. **Email Notification Service:** Implementation of an email notification system using AWS SES, SendGrid, or Nodemailer for subscription confirmation emails, renewal reminders, and new issue alerts.
53. **PDF Viewer Library Optimization:** Benchmarking and selection of the most performant React-based PDF viewer library (react-pdf, PDF.js, or PSPDFKit) to optimize the reading experience across devices and connection speeds.
54. **Mobile Application:** Development of a companion React Native mobile application for iOS and Android, enabling push notifications for new newspaper issues and an optimized mobile reading experience.
55. **Full-Text Search:** Implementation of a full-text newspaper title and content search feature using MongoDB Atlas Search or Elasticsearch to improve content discoverability.
56. **Analytics Dashboard:** An admin-facing analytics module for tracking subscription metrics, user engagement statistics, content popularity, and revenue reporting.
57. **Real-Time Notifications:** Push notification capability for new issue uploads via WebSockets or Firebase Cloud Messaging (FCM), enabling real-time alerts to subscribers when new content is available.
58. **Content Watermarking:** Server-side PDF watermarking with the subscriber's name or account ID before delivery, providing an additional layer of digital rights management (DRM) and deterring unauthorized content sharing.
59. **Multi-Language Support:** Internationalization (i18n) support for multiple languages to serve a broader demographic of newspaper publishers and subscribers.
60. **Auto-Scaling Infrastructure:** Migration from a fixed EC2 instance to AWS Elastic Beanstalk or Amazon ECS with auto-scaling policies to dynamically handle traffic spikes during peak news hours.

## Appendix A: Glossary

| Term       | Definition                                                                                                                                                                                                       |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MERN       | An acronym for the four technologies that form the application stack: MongoDB (database), Express.js (backend framework), React.js (frontend library), and Node.js (runtime environment).                        |
| SPA        | Single Page Application. A web application that loads a single HTML page and dynamically updates content as the user interacts, without full page reloads. The NewsVault frontend is implemented as a React SPA. |
| JWT        | JSON Web Token. A compact, digitally signed token used to securely transmit authentication claims between the client and server. Used in NewsVault to protect all premium API routes.                            |
| S3         | Amazon Simple Storage Service. A scalable cloud object storage service. All newspaper PDF issues in NewsVault are stored in a private S3 bucket.                                                                 |
| CloudFront | Amazon CloudFront. A globally distributed Content Delivery Network (CDN) that caches and delivers content from edge locations closest to the user, ensuring low-latency PDF delivery.                            |
| EC2        | Amazon Elastic Compute Cloud. A virtual server hosting service. The NewsVault Node.js backend is deployed on an Ubuntu EC2 t2.micro instance.                                                                    |
| Nginx      | A high-performance web server and reverse proxy. In NewsVault, Nginx listens on port 443 (HTTPS) and forwards requests to the Node.js application on port 5000.                                                  |
| CI/CD      | Continuous Integration and Continuous Deployment. An automated pipeline that builds, tests, and deploys application code on every commit. Implemented using Jenkins and Docker in NewsVault.                     |
| ODM        | Object Document Mapper. A library that provides schema-based modeling for application data stored in MongoDB. NewsVault uses Mongoose as its ODM.                                                                |
| bcrypt     | A password hashing library that applies a cost factor (salt rounds) to make brute-force attacks computationally expensive. All user passwords in NewsVault are hashed using bcrypt before storage.               |
| Signed URL | A time-limited, pre-authenticated URL generated by the server that grants temporary access to a specific private S3 object. Used in NewsVault to serve e-paper PDFs securely.                                    |
| CDN        | Content Delivery Network. A distributed network of servers that delivers content to users based on geographic proximity to reduce latency.                                                                       |
| REST API   | Representational State Transfer Application Programming Interface. An architectural style for designing networked applications. NewsVault uses a RESTful API for all client-server communication.                |

|        |                                                                                                                                                                            |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Docker | An open-source containerization platform that packages the application and its dependencies into portable containers, ensuring consistent behavior across environments.    |
| NoSQL  | A category of database management systems that do not use the traditional relational table-based model. MongoDB, used in NewsVault, is a document-oriented NoSQL database. |
| DRM    | Digital Rights Management. Techniques used to control how digital content is used and distributed. NewsVault uses Signed URLs and disabled downloads as DRM measures.      |

## Appendix B: To Be Determined List

The following items represent design decisions or features that have not yet been finalized and will be addressed in future iterations of the NewsVault platform:

61. Payment Gateway Integration: The specific payment processor (Stripe, Razorpay, or PayPal) to be integrated for handling subscription transactions has not yet been finalized.
62. PDF Viewer Library: The exact React library to be used for the in-browser e-paper reader (react-pdf, PDF.js, or PSPDFKit) is yet to be determined based on performance benchmarking.
63. Email Notification Service: The email service provider (AWS SES, SendGrid, or Nodemailer with SMTP) for sending subscription confirmation and renewal reminder emails is not yet selected.
64. Full-Text Search: Whether to implement a full-text newspaper title search feature using MongoDB Atlas Search or Elasticsearch is pending evaluation.
65. Mobile Application: The potential for a companion React Native mobile application for iOS and Android is noted but not included in the current release scope.
66. Analytics Dashboard: An admin-facing analytics module for tracking subscription metrics and user engagement may be considered for a future release.
67. Real-Time Notifications: Push notification capability for new issue uploads via WebSockets or Firebase Cloud Messaging (FCM) is to be evaluated for a subsequent version.
68. Content Watermarking: Server-side PDF watermarking with the subscriber's account details before delivery is under consideration as an additional DRM measure.

## Bibliography / References

69. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
70. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
71. MongoDB Inc. (2024). MongoDB Documentation. Retrieved from <https://www.mongodb.com/docs/>
72. Meta Platforms Inc. (2024). React.js Documentation. Retrieved from <https://react.dev/>
73. OpenJS Foundation. (2024). Express.js Documentation. Retrieved from <https://expressjs.com/>
74. OpenJS Foundation. (2024). Node.js Documentation. Retrieved from <https://nodejs.org/docs/>
75. Amazon Web Services. (2024). AWS EC2, S3, and CloudFront Developer Guides. Retrieved from <https://docs.aws.amazon.com/>
76. Jenkins Community. (2024). Jenkins Documentation. Retrieved from <https://www.jenkins.io/doc/>
77. Docker Inc. (2024). Docker Documentation. Retrieved from <https://docs.docker.com/>
78. Jones, M. B., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). IETF RFC 7519. Retrieved from <https://tools.ietf.org/html/rfc7519>
79. IEEE Computer Society. (2009). IEEE Standard 1016 – Software Design Descriptions. IEEE.
80. Nginx Inc. (2024). Nginx Documentation. Retrieved from <https://nginx.org/en/docs/>